01 — Python for Machine Learning

Oct 8, 2026 · @Ashishpreet

Module 01 of 17 · 🟢 Beginner · about 40 hours. You will learn the Python, NumPy, Pandas and Matplotlib needed for every later module, using manufacturing data throughout.

Why am I learning this?

Every step of a LoRA or QLoRA fine-tune is a Python script: loading a dataset is Pandas, a tokenised batch is an array, a training log is a plot. If Python is fluent, Modules 12–17 become about ideas instead of syntax.

What you learn here	Where it shows up when you fine-tune a Hugging Face model
Variables, types	Model names, learning rates, batch sizes in your training config
Lists and dicts	A chat message is `{"role": "user", "content": "..."}`; a dataset row is a dict
Loops and functions	The training loop; the `tokenize()` function you map over a dataset
Classes	Every model, tokenizer and Trainer is an object with methods
Files and JSON	Your training data lives in a JSONL file
NumPy	Token IDs, embeddings and logits are arrays
Pandas	Cleaning and inspecting your dataset before training
Matplotlib	Plotting training and validation loss to spot overfitting

Learning objectives

By the end of this module you can:

- [ ] Run Python in Google Colab and organise work in notebooks

- [ ] Use the core data types and the four collections confidently
- [ ] Write loops, conditions, functions, comprehensions and small classes
- [ ] Read and write CSV and JSON files and handle errors
- [ ] Do vectorised maths with NumPy arrays
- [ ] Load, clean, filter and group a table with Pandas
- [ ] Make clear scatter plots, histograms and line charts

Prerequisites

None beyond having seen a little Python. A Google account for Colab.

> **Important — the running example.** This module uses one manufacturing story throughout: a plant moulds plastic parts, and you analyse their weight, cycle time, cavities, material and piece price. Datasets marked *illustrative* are generated in code so you can run everything without downloading files.

1. Your workspace: Colab and notebooks

A notebook is a lab logbook that runs: text notes and code cells side by side, each cell's output printed right under it, so experiments stay readable and repeatable.

What it is. Google Colab is a free notebook in your browser, running on Google's machines, with a free GPU. Jupyter is the same idea on your own computer.

Why we need it. ML is experimental: you load data, look at it, try something, look again. A notebook keeps each step and its result together, like a trial sheet on the shop floor.

```
┌─ Cell 1 (code) ──────────────────┐
│ weight = 1.25                    │  ← Shift + Enter runs it
└──────────────────────────────────┘

┌─ Cell 2 (code) ──────────────────┐
│ print(weight * 140)              │
└──────────────────────────────────┘

  175.0                            ← output appears under the cell
┌─ Cell 3 (text) ──────────────────┐
│ ## Material cost check           │  ← notes in Markdown
└──────────────────────────────────┘
```

Do this now

1. Go to colab.research.google.com → **New notebook**.
2. Type `print("Hello, plant!")` and press **Shift + Enter**.
3. **+ Text** adds a note cell; **+ Code** adds a code cell.
4. **Runtime → Change runtime type → T4 GPU** when later modules need a GPU.
5. **File → Save a copy in Drive** so your work is kept.

> **Common mistake:** running cells out of order. A notebook remembers variables from every cell you have run, in the order you ran them. If results look strange, use **Runtime → Restart and run all** to run everything top to bottom.

> **Remember:** `pip install package` installs a library. In Colab, prefix it with `!` (`!pip install xgboost`); most ML libraries are already installed.

2. Variables and data types

A variable is a labelled bin on a rack: the label is the name, the contents are the value, and the type says what kind of thing is inside. Getting types right is the most common source of beginner bugs in ML.

Why we need it. Every column in a dataset has a type. Weight is a decimal, cavities a whole number, material a piece of text, "PPAP done" a yes/no. Models need numbers, so you must know what you have.

Type	Python name	Example	Manufacturing meaning
Integer	`int`	`cavities = 4`	Counts: cavities, shifts, defects
Decimal	`float`	`weight_kg = 1.25`	Measurements: weight, temperature, cycle time
Text	`str`	`material = "PP-TD20"`	Names, codes, defect descriptions
True/False	`bool`	`ppap_done = True`	Pass/fail, OK/NOK
Nothing	`None`	`last_audit = None`	A missing value

Python implementation: a should-cost calculation

```
# --- Inputs for one injection-moulded part ---
part_name   = "Bumper bracket"   # str
weight_kg   = 1.25               # float
rate_per_kg = 140                # int, ₹/kg for PP-TD20
cycle_s     = 32.5               # float, seconds per shot
cavities    = 4                  # int
machine_rate_per_h = 1800        # int, ₹/hour for a 350 T press

# --- Calculation ---
material_cost = weight_kg * rate_per_kg                      # 1.25 × 140 =
175.0
process_cost  = machine_rate_per_h / 3600 * cycle_s / cavities  # 0.5 ₹/s ×
32.5 s / 4 = 4.06
total = material_cost + process_cost

print(f"{part_name}: material ₹{material_cost:.2f}, process ₹
{process_cost:.2f}, total ₹{total:.2f}")
print(type(weight_kg), type(cavities), type(part_name))
```

Output:

```
Bumper bracket: material ₹175.00, process ₹4.06, total ₹179.06
<class 'float'> <class 'int'> <class 'str'>
```

Line by line: `=` stores a value under a name. `#` starts a comment Python ignores. `/` always gives a float. `f"...{value:.2f}..."` is an f-string: it inserts values into text, `.2f` means two decimals. `type()` tells you what a variable holds.

Operators you will use constantly

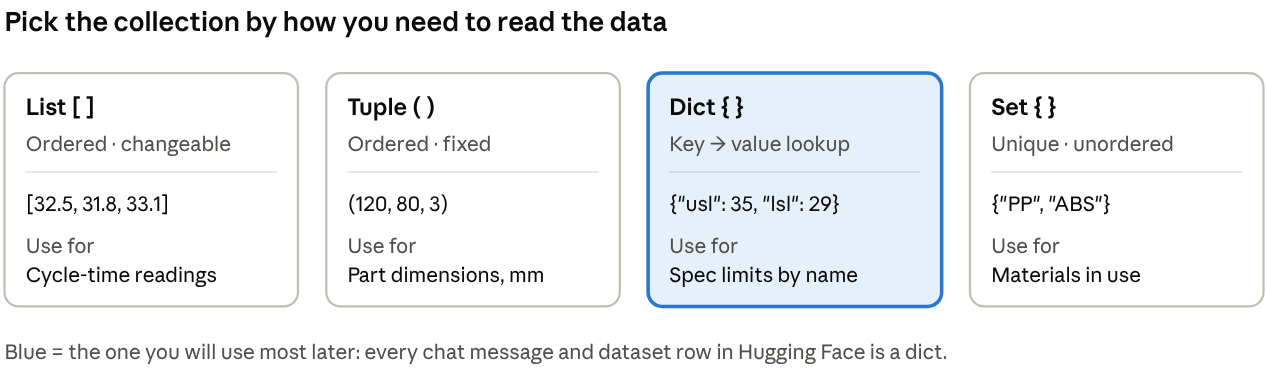
Operator	Meaning	Example	Result
`+ - * /`	Arithmetic	`7 / 2`	`3.5`
`//`	Whole-number division	`7 // 2`	`3`
`%`	Remainder	`7 % 2`	`1`
`**`	Power	`2 ** 3`	`8`
`== != < > <= >=`	Comparison, gives a bool	`cycle_s > 40`	`False`

Operator	Meaning	Example	Result
`and or not`	Combine bools	`ppap_done and cavities > 1`	`True`

> **Common mistake — text that looks like a number.** Data read from files often arrives as text: `"1.25" * 2` gives `"1.251.25"`, not 2.5. Convert first: `float("1.25") * 2`. In Pandas this shows up as a column of type `object` instead of `float64`.

3. Collections: list, tuple, dict, set

A collection holds many values under one name. Choose by how you will read the data back: by position (list, tuple), by name (dict), or just "is it in there?" (set).



Python's four built-in collections

Intuition. A list is a shift log you keep adding to. A tuple is a drawing dimension that must not change. A dict is a spec sheet: look up a value by its name. A set is the list of materials in stock, each counted once.

Python implementation

```
# List: ordered, changeable, positions start at 0
cycle_times = [32.5, 31.8, 33.1, 45.0]
cycle_times.append(32.2)           # add at the end
print(cycle_times[0], cycle_times[-1], len(cycle_times))   # 32.5 32.2 5
print(cycle_times[1:3])            # slice: positions 1 and 2 → [31.8, 33.1]

# Tuple: fixed once created
dims_mm = (120, 80, 3)
length, width, thickness = dims_mm   # unpacking
```

```
# Dict: key → value
spec = {"usl": 35.0, "lsl": 29.0, "target": 32.0}
print(spec["usl"])                  # 35.0
spec["unit"] = "s"                  # add a new key
for key, value in spec.items():
    print(key, value)

# A chat message for an LLM is just a dict; a conversation is a list of dicts
conversation = [
    {"role": "system", "content": "You are a moulding process engineer."},
    {"role": "user", "content": "Why do short shots happen?"},
]

# Set: unique values, fast membership test
materials = {"PP", "ABS", "PP", "PA6"}
print(materials)                    # {'PP', 'ABS', 'PA6'}  (duplicate
removed)
print("ABS" in materials)           # True
```

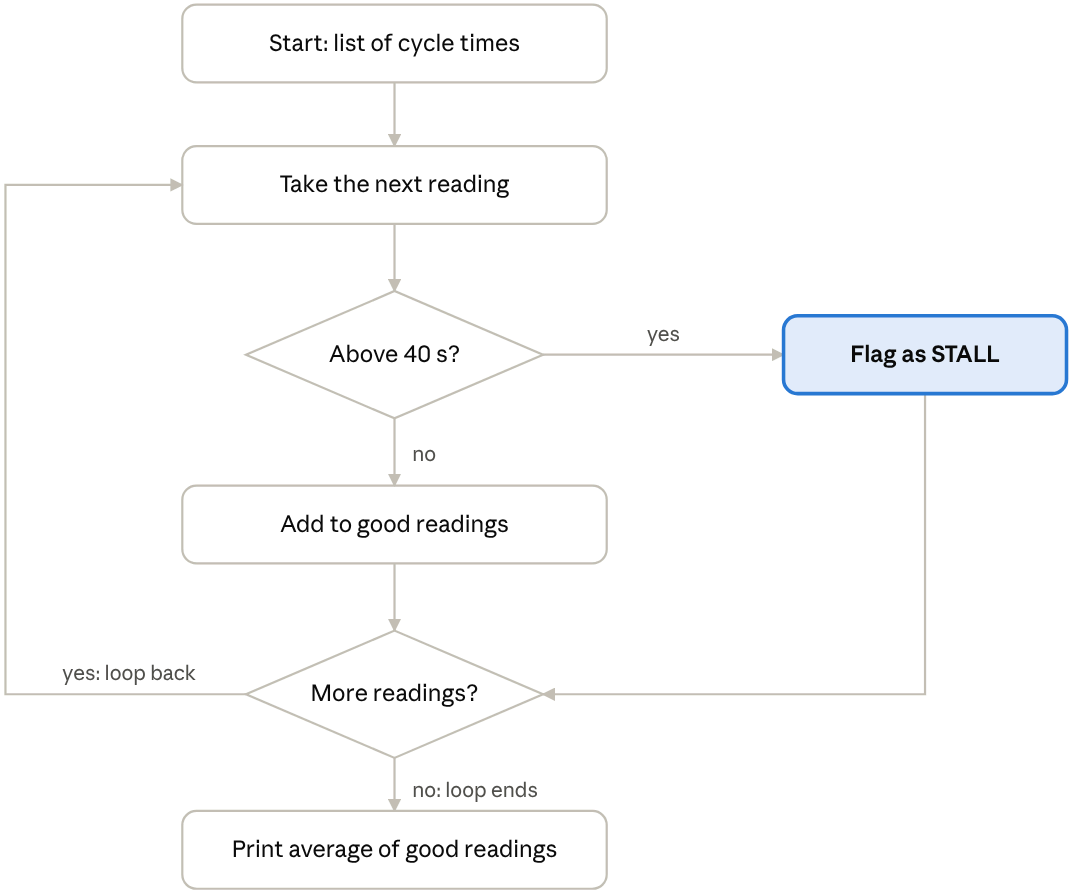
Need	Use	Why
A sequence you will add to	List	Ordered and changeable
A record that must not change	Tuple	Fixed; safe to pass around
Look up by name	Dict	Instant access by key
Remove duplicates, check membership	Set	Keeps one of each

> **Common mistake:** positions start at 0, not 1. `cycle_times[1]` is the *second* reading. `cycle_times[-1]` is the last.

4. Conditions and loops

A condition makes a decision (`if`); a loop repeats work for every item (`for`). Together they automate the checks an inspector does by hand, and the training loop in Module 08 is the same idea.

A loop checks every reading and flags the stalls



Flowchart of the cycle-time check below

Python implementation: the flowchart in code

```
cycle_times = [32.1, 31.8, 45.0, 32.6, 33.0, 52.3, 31.9]
good, stalls = [], []

for i, ct in enumerate(cycle_times):        # enumerate gives position and
value
    if ct > 40:                             # condition
        stalls.append(i)
        print(f"Shot {i}: {ct} s → STALL")
    elif ct < 28:
        print(f"Shot {i}: {ct} s → suspiciously fast, check sensor")
    else:
        good.append(ct)

avg = sum(good) / len(good)
print(f"Average of good shots: {avg:.2f} s; stalls at shots {stalls}")
```

Output:

```
Shot 2: 45.0 s → STALL
Shot 5: 52.3 s → STALL
Average of good shots: 32.28 s; stalls at shots [2, 5]
```

Line by line: the indented block under `for` runs once per item. `if / elif / else` checks conditions top to bottom and runs only the first true branch. Indentation (4 spaces) is how Python knows what belongs inside the loop; it is not decoration.

Other loop tools

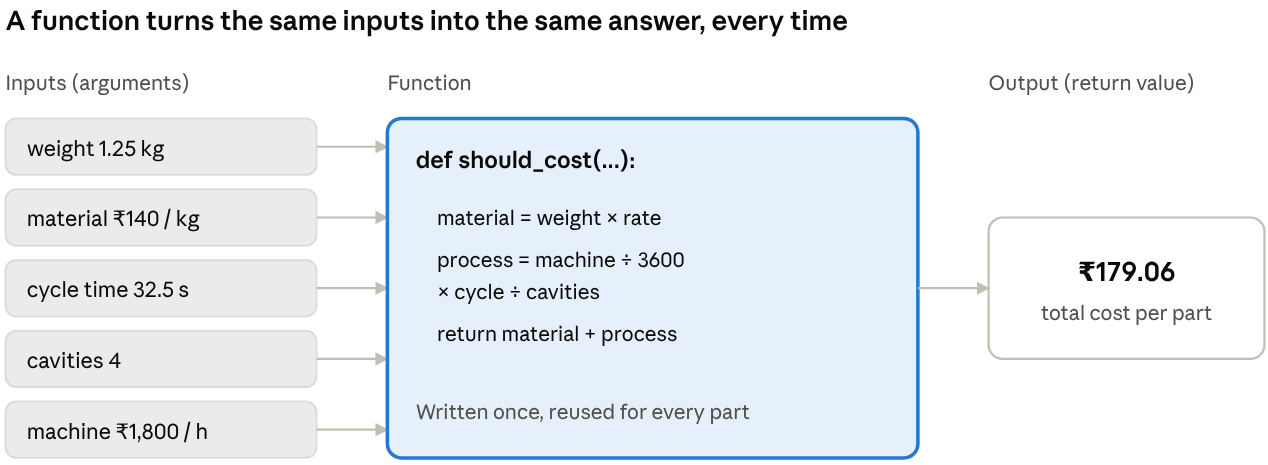
Tool	What it does	Example
`range(n)`	Numbers 0 to n−1	`for epoch in range(10):` (you will write this in every training loop)
`while`	Repeat while a condition holds	`while temp < setpoint: heat()`
`break`	Leave the loop early	Stop at the first critical alarm
`continue`	Skip to the next item	Skip blank readings

Tool	What it does	Example
`zip(a, b)`	Walk two lists together	`for part, wt in zip(parts, weights):`

> **Common mistake:** wrong indentation. If `avg = ...` were indented inside the loop, it would be recalculated after every reading. Python raises `IndentationError` only when the indentation is invalid, not when it is valid but wrong.

5. Functions, lambdas and comprehensions

A function packages a calculation so you write it once and reuse it everywhere, like a standard work instruction. Lambdas are one-line functions; comprehensions build a new list in one line.



The should-cost function from section 2, packaged for reuse

5.1 Functions

```
def should_cost(weight_kg, rate_per_kg, cycle_s, cavities,
machine_rate_per_h=1800):
    """Material + process cost for one moulded part, in ₹."""
    material = weight_kg * rate_per_kg
    process = machine_rate_per_h / 3600 * cycle_s / cavities
    return round(material + process, 2)

print(should_cost(1.25, 140, 32.5, 4))                          # 179.06
print(should_cost(0.80, 220, 28.0, 8, machine_rate_per_h=1500))  # 177.46
```

Line by line: `def` names the function and its inputs (parameters). `machine_rate_per_h=1800` is a default used when the caller does not give one. The `"""docstring"""` says what it does. `return` sends the answer back; without it the function returns `None`.

5.2 Lambda functions

A lambda is a nameless one-line function, handy for sorting and for Pandas.

```
to_minutes = lambda s: s / 60
print(to_minutes(90))                                  # 1.5

parts = [("grille", 0.80), ("bracket", 1.25), ("console", 2.10)]
print(sorted(parts, key=lambda p: p[1], reverse=True))   # heaviest first
```

5.3 List and dict comprehensions

A comprehension reads like a sentence: *"give me X for each item in the list, if a condition holds."*

```
cycle_times = [32.1, 31.8, 45.0, 32.6, 52.3]
stalls  = [ct for ct in cycle_times if ct > 40]          # [45.0, 52.3]
minutes = [round(ct / 60, 3) for ct in cycle_times]      # convert all
weights = {"grille": 0.80, "bracket": 1.25, "console": 2.10}
heavy   = {name: w for name, w in weights.items() if w > 1}   # {'bracket':
1.25, 'console': 2.1}
```

Write this loop…	…as this comprehension
`out = []` then `for x in xs: if x > 0: out.append(x * 2)`	`out = [x * 2 for x in xs if x > 0]`

> **Why it matters later:** in Module 12 you will write `def tokenize(batch): return tokenizer(batch["text"], truncation=True)` and apply it to a whole dataset with `dataset.map(tokenize)`. That is exactly this pattern.

> **Common mistake:** `print` instead of `return`. A function that only prints shows the number on screen but gives nothing back to use in the next calculation.

6. Classes, errors, files and modules

These four tools let your code model real equipment, survive bad data, read and save datasets, and reuse other people's code. You will use all four in every Hugging Face script.

6.1 Classes and objects

A **class** is a drawing; an **object** is a part made from it. The drawing defines attributes (dimensions) and methods (what the part can do); each object has its own values.

```
class Machine:
    def __init__(self, name, tonnage, rate_per_h):   # runs when an object is
created
        self.name = name                             # attributes: data
stored on the object
        self.tonnage = tonnage
        self.rate_per_h = rate_per_h

    def can_mould(self, required_tonnage):           # method: something the
object can do
        return self.tonnage >= required_tonnage

    def cost_per_part(self, cycle_s, cavities):
        return self.rate_per_h / 3600 * cycle_s / cavities

press = Machine("IMM-350", 350, 1800)                # create an object
print(press.can_mould(280))                          # True
print(round(press.cost_per_part(32.5, 4), 2))        # 4.06
```

> **Why it matters later:** `model = AutoModelForCausalLM.from_pretrained(...)` creates an object; `model.generate(...)` calls its method; `model.config` reads an attribute. Same pattern.

6.2 Handling errors

Real data is messy. `try / except` lets the program handle a problem instead of crashing.

```
readings = ["32.5", "31.9", "ERR", "33.4", ""]
clean = []
for r in readings:
    try:
        clean.append(float(r))           # may fail on 'ERR' or ''
    except ValueError:
        print(f"Skipped bad reading: {r!r}")
print(clean)                             # [32.5, 31.9, 33.4]
```

Error you will meet	Usual cause
`NameError`	Variable not defined, or the cell that defines it was not run

Error you will meet	Usual cause
`TypeError`	Wrong type, e.g. adding text to a number
`ValueError`	Right type, bad value, e.g. `float("ERR")`
`KeyError`	Dictionary or column name does not exist (check spelling)
`IndexError`	List position past the end
`ModuleNotFoundError`	Library not installed: `pip install` it

6.3 Files: CSV and JSON

```
import csv, json

# Write and read a CSV (comma-separated table)
rows = [["part", "weight_kg"], ["bracket", 1.25], ["grille", 0.80]]
with open("parts.csv", "w", newline="") as f:   # 'with' closes the file
automatically
    csv.writer(f).writerows(rows)
with open("parts.csv") as f:
    print(f.read())

# JSON: the format of Hugging Face configs and chat datasets
message = {"role": "user", "content": "What causes sink marks?"}
with open("sample.jsonl", "w") as f:              # JSONL = one JSON object
per line
    f.write(json.dumps(message) + "\n")
with open("sample.jsonl") as f:
    print(json.loads(f.readline())["content"])     # What causes sink marks?
```

6.4 Modules and packages

A module is a file of reusable code; a package is a folder of modules. `import` brings them in.

```
import math                    # whole module
from statistics import mean    # one function
import numpy as np             # with a short alias (the universal
convention)
print(math.sqrt(16), mean([31.8, 32.5, 33.1]), np.pi)
```

Install new packages with `pip install name` (`!pip install name` in Colab). On your own computer, keep each project in a **virtual environment** so package versions never clash: `python -m venv env`, then activate it before installing.

7. NumPy: arrays and vectorised maths

NumPy does maths on whole columns of numbers at once instead of one at a time, typically 10–100× faster than a Python loop. Neural networks, embeddings and model outputs are all NumPy-style arrays (PyTorch tensors behave the same way).

Intuition. A Python loop is an operator measuring parts one by one. A NumPy operation is a CMM program measuring the whole fixture in one pass.

```
 Python loop                          NumPy (vectorised)
 for each part:                       weights = [1.2, 0.8, 2.5, 1.9]
   cost = weight × rate               rates   = [140, 220,  95, 140]
   append cost                        cost    = weights * rates      ← one
line, all parts
                                                = [168, 176, 237.5, 266]
```

Shapes: the most important idea in this section

Object	Example	Shape	ML meaning
Scalar	`140`	`()`	One setting, e.g. learning rate
Vector (1-D)	`[1.2, 0.8, 2.5]`	`(3,)`	One feature column, or one data point
Matrix (2-D)	500 parts × 4 features	`(500, 4)`	A dataset; a layer's weights
3-D array	32 sentences × 128 tokens × 768 numbers	`(32, 128, 768)`	A batch of token embeddings in a Transformer

Python implementation

```
import numpy as np

weights = np.array([1.2, 0.8, 2.5, 1.9])        # kg
rates   = np.array([140, 220, 95, 140])         # ₹/kg
material_cost = weights * rates                 # element by element
print(material_cost)                            # [168.  176.  237.5 266. ]
print(material_cost.sum(), material_cost.mean().round(1),
material_cost.max())

# A dataset as a matrix: rows = parts, columns = features
X = np.array([[1.2, 32.0, 4],
              [0.8, 28.5, 8],
              [2.5, 45.0, 2]])                  # weight, cycle time,
cavities
print(X.shape)                                  # (3, 3)
print(X[:, 1])                                  # all rows, column 1 → cycle
times
print(X[X[:, 0] > 1.0])                         # parts heavier than 1 kg
(boolean mask)
print(X.mean(axis=0))                           # average of each column

# Matrix multiplication: the core operation of every neural network layer
w = np.array([140, 1.5, -5])                    # one weight per feature
print(X @ w)                                    # price-like score for each
part

# Random data for experiments (reproducible with a seed)
rng = np.random.default_rng(42)
cycle_times = rng.normal(loc=32, scale=1.5, size=1000)   # mean 32 s, sd 1.5
s
print(cycle_times[:5].round(2), cycle_times.std().round(2))
```

Line by line, the key ideas: `np.array` makes an array. `*` between arrays multiplies matching elements. `X[:, 1]` reads "all rows, column 1". `X[X[:, 0] > 1.0]` keeps rows where the condition is True. `axis=0` means "down the rows" (one result per column). `@` is matrix multiplication. `default_rng(42)` gives the same random numbers every run.

Manufacturing example: Cpk in three lines

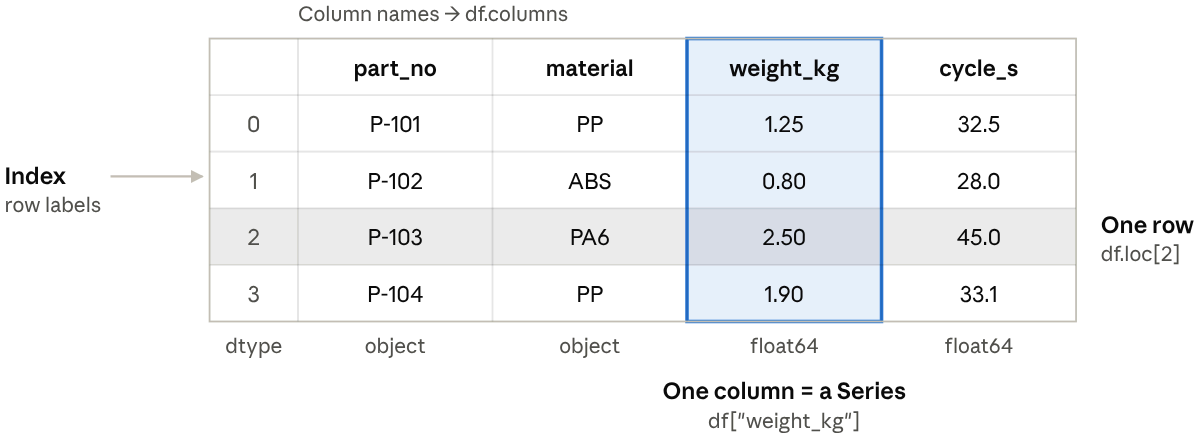
```
usl, lsl = 35.0, 29.0                          # spec limits on cycle time,
s
mu, sigma = cycle_times.mean(), cycle_times.std()
cpk = min(usl - mu, mu - lsl) / (3 * sigma)
print(f"Cpk = {cpk:.2f}")                      # ≈ 0.67 for sd 1.5: not
capable (target ⩾ 1.33)
```

> **Common mistake — shape mismatch.** `(3, 3) @ (2,)` fails with "shapes not aligned". Print `.shape` of every array before combining them; it is the first debugging step in every ML error you will meet.

8. Pandas: tables you can program

Pandas is Excel you control with code: load a table, inspect it, clean it, filter it, group it, in a few lines that you can rerun on next month's data. You will prepare every fine-tuning dataset with it.

A DataFrame is named columns that share one index



Anatomy of a Pandas DataFrame · 4 illustrative parts

Key vocabulary. A **DataFrame** is the whole table. Each column is a **Series** with one data type (`dtype`). The **index** labels the rows. Text columns show as `object` ; numbers as `int64` or `float64` .

8.1 Load and inspect

```
import numpy as np
import pandas as pd

# Illustrative dataset: 300 moulded parts (replace with
pd.read_csv("your_file.csv"))
rng = np.random.default_rng(7)
n = 300
df = pd.DataFrame({
    "part_no":  [f"P-{100 + i}" for i in range(n)],
    "material": rng.choice(["PP", "ABS", "PA6"], n, p=[0.5, 0.3, 0.2]),
    "weight_kg": rng.uniform(0.3, 2.8, n).round(2),
    "cycle_s":  rng.normal(32, 1.5, n).round(1),
    "cavities": rng.choice([1, 2, 4, 8], n),
})
rate = df["material"].map({"PP": 140, "ABS": 210, "PA6": 260})   # ₹/kg by
material
df["price"] = (df["weight_kg"] * rate + 1800 / 3600 * df["cycle_s"] /
df["cavities"]
               + rng.normal(0, 8, n)).round(1)
df.loc[rng.choice(n, 12, replace=False), "weight_kg"] = np.nan    # realistic
gaps

df.head()          # first 5 rows
df.info()          # columns, dtypes, non-null counts
df.describe()      # count, mean, std, min, quartiles, max
df["material"].value_counts()
```

8.2 Select, filter, add columns

```
df["weight_kg"]                                 # one column → Series
df[["part_no", "price"]]                        # several columns → DataFrame
df.loc[2]                                       # row with index label 2
df.iloc[0:5]                                    # first five rows by position
df[df["cycle_s"] > 35]                          # filter rows by a condition
df[(df["material"] == "PP") & (df["cavities"] >= 4)]   # combine with & and
|, brackets required
df["price_per_kg"] = df["price"] / df["weight_kg"]     # vectorised new
column
```

8.3 Clean

```
print(df.isna().sum())                                          # missing values
per column
df["weight_kg"] = df["weight_kg"].fillna(df["weight_kg"].median())
df = df.drop_duplicates()
df["cycle_s"] = pd.to_numeric(df["cycle_s"], errors="coerce")  # text →
number, bad values → NaN
```

8.4 Group and summarise

The single most useful Pandas skill: answer "per material, per machine, per shift" questions in one line.

```
summary = (df.groupby("material")
             .agg(parts=("part_no", "count"),
                  avg_weight=("weight_kg", "mean"),
                  avg_price=("price", "mean"))
             .round(2)
             .sort_values("avg_price", ascending=False))
print(summary)
df.to_csv("parts_clean.csv", index=False)     # save for the next module
```

Task	Excel	Pandas
Filter rows	AutoFilter	`df[df["c"] > 5]`
New column	Formula dragged down	`df["new"] = df["a"] * df["b"]`
Pivot / summary	PivotTable	`df.groupby("g").agg(...)` or `df.pivot_table(...)`
VLOOKUP	VLOOKUP	`df.merge(other, on="part_no")`
Remove duplicates	Data → Remove duplicates	`df.drop_duplicates()`

> **Common mistake:** `df[df["a"] > 1 and df["b"] < 5]` raises an error. Use `&` (and) and `|` (or), with each condition in brackets.

9. Visualisation with Matplotlib

Always plot before you model: a chart shows relationships, spread and outliers that a table of numbers hides. Two chart types cover most early work: the scatter plot (how does Y move with X?) and the histogram (how is one variable spread?).

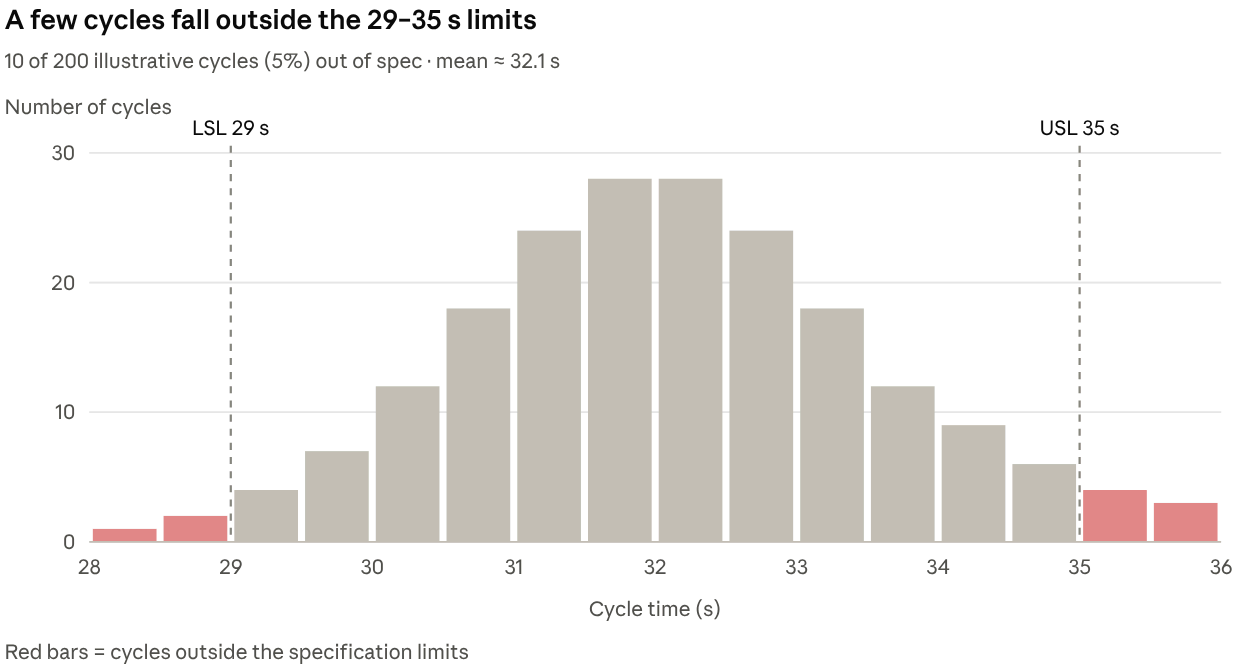
9.1 Scatter plot: does price follow weight?



Illustrative data · 24 moulded parts · line fitted by least squares

Engineering interpretation. The points lie close to a straight line, so weight explains most of the price: roughly ₹150 per extra kg, which matches a material rate of about ₹140–150/kg. A part sitting far *above* the line is a candidate for a VA/VE or price negotiation; far below, check whether the quote missed something. This line is your first machine-learning model; Module 04 shows how it is fitted.

9.2 Histogram: how stable is the cycle time?



Illustrative data · 200 simulated cycles, mean 32 s, sd ≈ 1.5 s

Engineering interpretation. The process is centred near 32 s, but the spread is wide relative to the 29–35 s window, so about 1 cycle in 20 is out of spec. With mean 32 s and sd 1.5 s, Cpk ≈ 0.67: not capable (target ⩾ 1.33). The fix is to reduce variation, not to move the mean.

9.3 The code that makes these charts

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)

# --- Scatter with a fitted line ---
weight = rng.uniform(0.4, 2.8, 24)
price = 60 + 150 * weight + rng.normal(0, 8, 24)        # illustrative
relationship + noise
m, b = np.polyfit(weight, price, deg=1)                 # least-squares slope
and intercept

fig, ax = plt.subplots(figsize=(7, 4))
ax.scatter(weight, price, color="grey", label="parts")
xs = np.linspace(0.3, 2.9, 50)
```

```
ax.plot(xs, b + m * xs, color="tab:blue", linewidth=2, label=f"price =
{b:.0f} + {m:.0f} × weight")
ax.set_xlabel("Part weight (kg)"); ax.set_ylabel("Piece price (₹)")
ax.set_title("Price vs weight"); ax.legend(); ax.grid(alpha=0.3)
plt.show()

# --- Histogram with spec limits ---
cycle = rng.normal(32, 1.5, 200)
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(cycle, bins=np.arange(28, 36.5, 0.5), color="grey",
edgecolor="white")
for limit, name in [(29, "LSL"), (35, "USL")]:
    ax.axvline(limit, color="red", linestyle="--")
    ax.text(limit, ax.get_ylim()[1] * 0.95, name, ha="center")
out = ((cycle < 29) | (cycle > 35)).sum()
ax.set_title(f"{out} of {len(cycle)} cycles out of spec")
ax.set_xlabel("Cycle time (s)"); ax.set_ylabel("Number of cycles")
plt.show()
```

Line by line, the essentials: `plt.subplots()` creates a figure (`fig`) and drawing area (`ax`). `ax.scatter`, `ax.hist`, `ax.plot`, `ax.axvline` add marks. `np.polyfit(x, y, 1)` fits a straight line. Always label both axes with units.

Question	Chart
How does Y change with X?	Scatter (`ax.scatter`)
How is one variable spread? Any outliers?	Histogram (`ax.hist`) or box plot
How does a value change over time (e.g. training loss)?	Line (`ax.plot`)
How do categories compare?	Vertical bar / column (`ax.bar`)
How do many variables relate?	Correlation heatmap (`seaborn.heatmap`)

> **Remember:** in Module 13 the most important chart you will make is training loss and validation loss against steps, a line chart. If validation loss rises while training loss falls, the model is overfitting.

Common mistakes and interview questions

Most Python-for-ML bugs come from six habits; fix these and you avoid most early frustration.

Mistake	Symptom	Fix
Running notebook cells out of order	Results change mysteriously; `NameError`	Runtime → Restart and run all
Numbers stored as text	Sums fail or concatenate; column type `object`	`float()`, or `pd.to_numeric(df["c"], errors="coerce")`
Looping over arrays or DataFrames	Code takes minutes instead of milliseconds	Vectorise: `df["a"] * df["b"]`
Single vs double brackets in Pandas	scikit-learn complains about 1-D input	`df["col"]` is a Series (1-D); `df[["col"]]` is a table (2-D)
Modifying a list while looping over it	Items skipped silently	Build a new list with a comprehension
Ignoring shapes	"shapes not aligned" errors	Print `.shape` before every combine

Interview questions

1. What is the difference between a list and a tuple? When would you use each?
2. Why is NumPy faster than a Python loop?
3. What does `axis=0` mean in `X.mean(axis=0)`?
4. How do you find and handle missing values in a Pandas DataFrame?
5. What is the difference between `df.loc` and `df.iloc`?
6. Why use a virtual environment?
7. Explain a list comprehension and give an example from your own work.

Exercises and mini project

Do the exercises in order; each takes 15–30 minutes. Write the code yourself before looking anything up.

Exercises

1. **OEE function.** Write `oee(availability, performance, quality)` taking fractions (0.9, 0.95, 0.99) and returning OEE as a percentage rounded to 1 decimal. Test: `oee(0.9, 0.95, 0.99)` → `84.6`.
2. **Parts dictionary.** Make a dict of 5 parts → weight. Use a comprehension to list parts over 1 kg, sorted heaviest first.
3. **Clean readings.** From `["32.1", "31.8", "N/A", "45.0", "32.6", ""]`, write code that skips bad values, converts the rest, and flags anything above 40 s as `"STALL"`.
4. **Machine class.** Add a method `annual_capacity(cycle_s, cavities, hours_per_year=6000)` to the `Machine` class that returns parts per year.
5. **NumPy Cpk.** Generate 500 shaft diameters with mean 20.02 mm and sd 0.01 mm; compute Cpk for spec 20.00 ± 0.05 mm.
6. **Pandas.** On the parts DataFrame from section 8, add `total_cost` and find the cheapest material per kg on average.
7. **Plot.** Plot a histogram of your 500 diameters with vertical lines at LSL and USL.

Mini project — Should-cost explorer

Goal: a notebook that answers "Which parts are priced above their should-cost, and by how much?"

1. Generate 300 illustrative parts with NumPy: weight, cycle time, cavities, material, and a quoted price = should-cost × a random factor between 0.9 and 1.4.
2. Put them in a DataFrame; compute should-cost (material + process) with vectorised columns.
3. Add `gap_pct = (quoted − should) / should × 100`.
4. Show the 10 parts with the highest gap, the average gap by material, and how many parts are more than 20% over.
5. Make two charts: quoted vs should-cost scatter with a y = x reference line, and a histogram of `gap_pct`.
6. Write three sentences of findings, as you would in a VA/VE review.

Done when: the notebook runs top to bottom after "Restart and run all" and a colleague could read your findings without you explaining them.

Quiz

Answer all ten on paper before checking the answers at the end of this document. Eight or more correct means you are ready to move on.

1. **Concept.** What type does `7 / 2` return, and what is its value?
2. **Concept.** Which collection would you use to store each part number with its weight, and why?
3. **Prediction.** What does `[c * 2 for c in [1, 2, 3, 4] if c % 2 == 0]` produce?
4. **Prediction.** `X` has shape (500, 4). What is the shape of `X.mean(axis=0)`?
5. **Maths.** Weights `[1.0, 2.0]` kg at rates `[100, 150]` ₹/kg. What does `(w * r).sum()` return?
6. **Debugging.** `df["weight_kg"].sum()` returns `"1.20.82.5"`. What went wrong and how do you fix it?
7. **Debugging.** A cell raises `NameError: name 'df' is not defined`, but you defined `df` earlier. Most likely cause?
8. **Coding.** Write one Pandas line that keeps only rows where material is `"PP"` and cycle time is under 35 s.
9. **Coding.** Write a lambda that converts seconds to minutes.
10. **Decision.** A colleague loops over 2 million sensor readings in Python to compute a moving average and it takes 4 minutes. What do you suggest?

Summary, cheat sheet and next steps

You now have the Python toolkit every later module assumes: core types and collections, control flow, functions and classes, files, NumPy arrays, Pandas tables and Matplotlib charts.

Summary

- Variables hold typed values; check types when data comes from files.
- Lists (ordered, changeable), tuples (fixed), dicts (key → value), sets (unique).
- Loops and conditions automate checks; comprehensions do it in one line.
- Functions package logic; classes model things (and every Hugging Face model is one).
- NumPy: whole-array maths, and shapes matter more than anything.
- Pandas: load → inspect → clean → filter → group.

- Plot before you model.

Cheat sheet

```
# Core
x = 1.25; type(x); f"{x:.2f}"; int("4"); float("1.2"); str(4)
parts = ["a", "b"]; parts.append("c"); parts[0]; parts[-1]; len(parts)
spec = {"usl": 35, "lsl": 29}; spec["usl"]; spec.get("target", 32);
spec.items()
[x * 2 for x in nums if x > 0]; lambda s: s / 60
for i, v in enumerate(vals): ...; while cond: ...; if a: ... elif b: ...
else: ...
try: ... except ValueError as e: ...
with open("f.json") as f: data = json.load(f)

# NumPy
import numpy as np
a = np.array([...]); a.shape; a.mean(); a.std(); a[a > 0]; X[:, 1];
X.mean(axis=0); X @ w
rng = np.random.default_rng(42); rng.normal(32, 1.5, 1000)

# Pandas
import pandas as pd
df = pd.read_csv("f.csv"); df.info(); df.describe(); df.head()
df.isna().sum(); df.drop_duplicates(); df["c"].fillna(df["c"].median())
df[df["c"] > 5]; df.loc[mask, ["a", "b"]]; df.groupby("g")["y"].mean()
df["new"] = df["a"] * df["b"]; df.sort_values("y", ascending=False);
df.to_csv("out.csv", index=False)

# Matplotlib
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 4))
ax.scatter(x, y); ax.hist(v, bins=30); ax.plot(x, y)
ax.set_xlabel("..."); ax.set_ylabel("..."); ax.set_title("..."); plt.show()
```

Further learning

- *Python for Data Analysis* (3rd ed.) by Wes McKinney, creator of Pandas; free online.
- Kaggle Learn: free short courses "Python" and "Pandas" with in-browser exercises.
- The official NumPy "absolute beginners" guide on numpy.org.

Can I move on?

- [x] ~~I understand types, collections, loops, functions and classes~~
- [ ] I can explain why NumPy is faster than a loop, and what a shape is
- [ ] I can load, clean, filter and group a CSV with Pandas from memory
- [ ] I can read a scatter plot and a histogram and say what they show
- [ ] I finished the should-cost explorer mini project

Next: Module 02 — Mathematics for Machine Learning: vectors, matrices, derivatives and gradients, taught with the same manufacturing examples.

Quiz answers

1. `float`, `3.5`.
2. A dict: look up a weight by part number, e.g. `weights["P-1001"]`.
3. `[4, 8]`.
4. `(4,)`: one mean per column.
5. `1.0×100 + 2.0×150 = 400.0`.
6. The column is text, so `+` joins strings. Fix with `pd.to_numeric(df["weight_kg"], errors="coerce")`.
7. The cell that defines `df` was not run in this session (or the runtime restarted).
8. `df[(df["material"] == "PP") & (df["cycle_s"] < 35)]`.
9. `to_min = lambda s: s / 60`.
10. Vectorise: load into NumPy or Pandas and use `pd.Series(x).rolling(window).mean()`, which runs in well under a second.